C Programming Language

1. Introduction to Programming

1. **Meaning / Definition of Programming :**
   Programming is the process of designing, writing, testing and maintaining a set of instructions (programs) that a computer can execute to perform specific tasks or solve problems.
2. **What is a Program ?**
   A program is a set of step-by-step instructions written in a programming language to accomplish a particular task.
3. **Need of Programming in Computers :**
   - → Computers can understand only instructions.
   - → Programming provides those instructions.
   - → It tells the computer what operations to perform and how to perform.
   - → Helps in solving complex problems accurately and quickly.
   - → It is essential for system software, application software and modern technologies.
4. **Characteristics of a Good Program :**
   - Correctness : Produces correct results for all valid inputs.
   - Simplicity : Easy to design, code and understand.
   - Efficiency : Uses minimum time and memory.
   - Readability : Easy to read and understand by others.
   - Maintainability : Easy to modify, test and update.
5. **Types of Programming Languages :**
   (i) Machine Language : Low-level language. Consists of 0s and 1s. Directly understood by the computer. Difficult to write.
   (ii) Assembily Language : Uses mnemonics (e.g., MOV, ADD). Needs an assembler to convert into machine code.
   (iii) High-level Language : Close to human language (e.g., C, Java, Python). Easy to write, understand and maintain. Portable.
6. **Why we learn C Language ?**
   - → C is simple, structured and powerful.
   - → It is a middle-level language (supports both low-level and high-level features).
   - → It is portable, fast and widely used in system programming.
   - → It forms the base for many modern languages (C++, Java, Python, etc.).
7. **Exam-Oriented Note :**
   > Programming is the process of writing instructions for a computer to solve a problem.
8. **Uses of Programming in Real Life :**
   - Developing software and mobile apps
   - Banking and billing systems
   - Games and animations
   - Communication and networking
   - Scientific calculations and simulations
   - Artificial Intelligence and automation
9. **Programming Process (Flow) :**

Problem	→	Algorithm	→	Program	→	Output
Real life situation to be solved.		Step-by-step solution in logical order.		Coding the algorithm in a programming language.		Result produced by executing the program.

10. **Important Points / Revision :**
    (i) A program is a set of instructions written to solve a problem.
    (ii) Good programming requires logic, accuracy and clarity.
    (iii) C language is foundational and widely used in computer applications.

C Programming Language

2. Problem Solving and Algorithms

1. **Meaning of Problem Solving in Programming :**
   Problem solving in programming is the process of finding the best solution to a given problem using a computer. It involves understanding the problem, developing the logic and writing a program to get the required result.

2. **Steps in Problem Solving :**
   - → (1) Understanding the Problem : Read the problem carefully and identify what is to be done.
   - → (2) Input / Output Analysis : Determine the inputs required and the expected outputs.
   - → (3) Logic Development : Develop the logic or approach to solve the problem.
   - → (4) Algorithm : Write the step-by-step solution in simple language.
   - → (5) Coding : Convert the algorithm into a program using a programming language.
   - → (6) Testing : Run the program with test data and check results.
   - → (7) Debugging : Find and remove errors in the program.
   - → (8) Documentation : Write comments and prepare necessary documents for future reference.

3. **Definition of Algorithm :**
   An algorithm is a finite set of well-defined step-by-step instructions to solve a problem. It gives the logic of solution in a systematic manner.

4. **Characteristics of a Good Algorithm :**
   - • Finiteness : It must have a finite number of steps and must stop after completing them.
   - • Definiteness : Each step must be precise, clear and unambiguous.
   - • Input : It should have zero or more well-defined inputs.
   - • Output : It should produce at least one well-defined output.
   - • Effectiveness : Each step must be basic and feasible to perform in finite time.

5. **Advantages of Algorithms :**
   - → Helps in understanding the problem clearly.
   - → Acts as a blueprint before coding.
   - → Makes program writing easier and faster.
   - → Helps in debugging and modifying programs.
   - → Saves time and improves program quality.

6. **Example Algorithm : To Find the Sum of Two Numbers**
   (1) Start.
   (2) Declare three variables : num1, num2 and sum.
   (3) Read num1 and num2.
   (4) Calculate sum ← num1 + num2.
   (5) Display sum.
   (6) Stop.

7. **Exam Note :**

   > An algorithm contains the logic of the solution, while a program is the coded implementation of that logic in a programming language.

8. **Important Points / Revision :**
   (i) Always understand the problem fully before writing the algorithm.
   (ii) A good algorithm leads to a simple, correct and efficient program.
   (iii) Algorithm is language-independent and can be written in simple English.

C Programming Language.

3. Flowchart

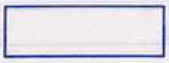
1. Definition of Flowchart :

A flowchart is a graphical representation of an algorithm or process. It uses standard symbols connected by arrows to show the flow of control from start to finish.

2. Importance / Advantages of Flowcharts :

- Provides a clear and simple visual representation of a process.
- Helps in planning and designing the program logic.
- Easy to understand and communicate with others.
- Helps in debugging by tracing the logic flow.
- Acts as a guide during coding and testing.
- Saves time and improves the quality of the program.

3. Common Flowchart Symbols :

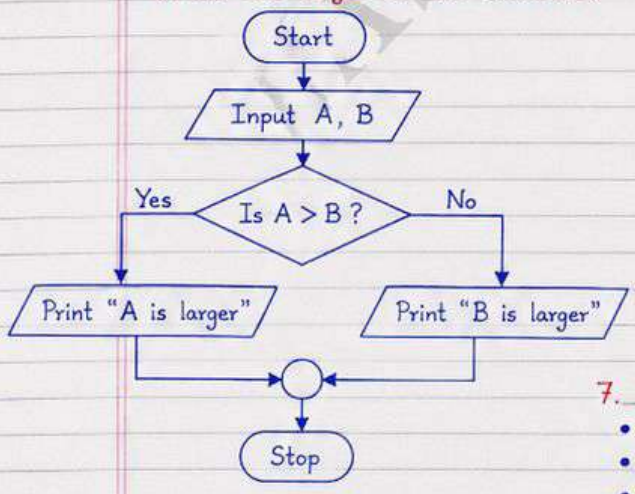
Symbol	Name	Purpose / Use
	Start / Stop	Indicates the beginning or end of a program.
	Input / Output	Used for input from user or output to display / print.
	Process	Represents a processing step or computation.
	Decision	Used for a decision or condition (Yes / No).
	Arrow / Flow line	Shows the direction of flow of control.
	Connector	Connects flow lines on the same page (A) or different pages (A).

4. Basic Rules for Drawing a Flowchart :

- A flowchart must have exactly one Start symbol and one Stop symbol.
- Flow is from top to bottom and from left to right.
- Each symbol must have a clear meaning.
- Flow lines (arrows) must not cross each other.
- Every decision must have two outgoing paths (Yes and No).
- Use connectors when the flowchart becomes long.

5. Example Flowchart :

(Find the larger of two numbers)



6. Algorithm vs Flowchart :

Basis	Algorithm	Flowchart
Meaning	Step-by-step textual description.	Graphical representation of steps.
Representation	Written in words.	Uses standard symbols.
Readability	May be long and confusing.	Easy to understand at a glance.
Modification	Time-consuming.	Easy to modify.
Best Use	For detailed logic.	For overall program design and flow.

7. Important Points / Revision :

- Flowchart is a graphical tool used before coding.
- Use correct symbols and proper flow direction.
- Always test your flowchart with different cases.
- Flowcharts help in understanding, coding and debugging.
- Learn symbols and practice drawing different type of flowcharts.

C Programming Language

4. History and Development of C

1. **Origin of C Language :**
   - C language was developed as a system programming language.
   - It emerged from the need for a portable and efficient language for writing system software.
   - The invention belongs to the early 1970s at Bell Laboratories.
2. **Dennis Ritchie and Bell Laboratories :**
   - Dennis MacAlistair Ritchie, a computer scientist at Bell Laboratories (AT&T), designed and implemented the C language.
   - He wanted a language that could be used to write the UNIX operating system.
   - Ritchie is also known as the "Father of C Language".
3. **Evolution : From BCPL to B to C**
   - BCPL (Basic Combined Programming Language) was developed by Martin Richards in the late 1960s.
   - BCPL influenced the development of B language.
   - B was a simplified, more efficient language developed by Ritchie around 1970.
   - C evolved from B by adding data types, structures, functions and more powerful features.
4. **Connection with UNIX Operating System :**
   - C was primarily developed to write the UNIX operating system.
   - Because C could interact directly with system hardware and manage memory efficiently.
   - The portability of UNIX across different machines was possible largely due to C.
5. **Year of Development :**
   - C language was developed around 1972.
6. **Standardization :**
   - ANSI C : In 1989, the American National Standards Institute (ANSI) standardized C as ANSI C (also called C89).
   - ISO C : In 1990, the International Organization for Standardization (ISO) adopted the ANSI C standard as ISO/IEC 9899:1990.
   - Later updates : C99 (1999), C11 (2011), C17 (2018) for improvements and new features.
7. **Why C Became Popular :**
   - Simple, structured and mid-level language.
   - Efficient and fast execution.
   - Portable — can be compiled on many platforms.
   - Provides low-level features like pointers and memory access.
   - Wide range of applications — system software, compilers, databases, embedded systems, games, etc..
8. **Evolution Timeline :**

BCPL	→	B	→	C	→	ANSI C
Late 1960s Designed by Martin Richards.		Around 1970 Developed by Dennis Ritchie.		Around 1972 Developed at Bell Laboratories.		1989 (C89) Standardized by ANSI.

9. **Important Questions / Revision Points :**
   (i) Who developed C language and at which organization?
   (ii) How did C evolve from BCPL?
   (iii) Why was C developed?

C Programming Language

5. Features and Applications of C

1. Main Features of C Language :

- **Simple** : Easy to learn and understand. It has a small set of keywords and simple syntax.
- **Structured** : Supports structured programming with functions, loops and conditionals.
- **Middle-level** : Combination of low-level features (hardware access) and high-level features (modularity, functions).
- **Portable** : C programs can be compiled and run on different systems with little or no modification.
- **Efficient** : Provides efficient code and optimal use of memory and CPU.
- **Rich Library** : Provides a large number of standard library functions.
- **Fast Execution** : C programs compile to machine code and run very fast.
- **Modularity** : Supports modular programming using functions and header files.
- **Pointers Support** : Supports pointers which allows direct memory access and dynamic data structures.

2. Applications of C Language :

- **Operating Systems** : Windows, Linux, UNIX kernel, etc.
- **Embedded Systems** : Used in microcontrollers and real-time systems.
- **Compilers and Interpreters** : Many compilers and language processors are written in C.
- **Device Drivers** : C is widely used for writing system and device drivers.
- **Games and Multimedia** : Used to develop games, graphics engines and multimedia tools.
- **Utilities and Tools** : Text editors, file managers, antivirus, compression tools, etc.
- **Scientific and Engineering Software** : Used in mathematical simulations, CAD, etc.

3. Advantages of C Language :

- Easy to learn and forms the base for other programming languages.
- Efficient and fast execution.
- Portable - can be used on many platforms.
- Provides direct access to memory using pointers.
- Large collection of standard libraries and user-defined functions.
- Suitable for system programming and embedded systems.
- Structured and modular - supports top-down programming approach.

4. Limitations of C Language :

- Does not support Object-Oriented Programming (OOP) concepts.
- Does not support exception handling.
- No built-in string type (uses character arrays).
- Manual memory management - chances of memory leaks, dangling pointers.
- Complex syntax compared to some modern languages.
- Limited security features.

5. Advantages vs Limitations of C

Advantages	Limitations
• Simple, structured and easy to learn. • Efficient, fast and close to hardware. • Portable across many platforms. • Rich library support. • Ideal for system and embedded programming.	• No support for OOP features. • No exception handling mechanism. • Manual memory management. • No built-in string type. • Less secure and more error-prone.

6. Important Points / Revision :

(i) Write any five features of C language.
(ii) List the applications of C.
(iii) State the advantages and limitations of C language.

C Programming Language

6. Structure of a C Program

1. General Structure of a C Program :

A C program is organized into different sections. These sections together form a complete program and each section has a specific purpose.

2. Different Sections of a C Program :

Section		Purpose
(i) Documentation Section	→	Contains comments about the program (purpose, author, date, etc.).
(ii) Link / Preprocessor Section	→	Includes header files using #include.
(iii) Definition Section	→	Contains symbolic constants using #define.
(iv) Global Declaration Section	→	Declares global variables and function prototypes.
(v) main() Function Section	→	Contains the main() function. Execution of program begins from main().
(vi) User-defined Function Section	→	Contains user-defined functions called by main() or by other functions.

3. Importance of main() in C :

- Every C program must have a main() function.
- It is the starting point of program execution.
- Control always begins from main() and returns to the operating system when main() ends.
- It may call other functions defined in the program.

4. Skeleton (Format) of a Simple C Program :

```
/* 1. Documentation Section */
// Program to add two numbers
// Author : Your Name
// Date   : __/__/____
#include <stdio.h>          // 2. Link / Preprocessor Section
#define PI 3.1416           // 3. Definition Section
int g = 10;                 // 4. Global Declaration Section
int add(int a, int b);      // Function Prototype
int main()                  // 5. main() Function Section
{
    int x, y, sum;
    printf("Enter two numbers: ");
    scanf("%d %d", &x, &y);
    sum = add(x, y);
    printf("Sum = %d\n", sum);
    return 0;
}
// 6. User-defined Function Section
int add(int a, int b)
{
    return (a + b);
}
```

5. Short Notes :

(i) **Header Files :**
Header files (.h files) contain predefined function declarations, macros and constants.
Example : stdio.h, conio.h, math.h, string.h, etc.
They are included using #include.

(ii) **Semicolons (;) :**
Semicolon is used to end every statement in C.
It tells the compiler that the statement is complete.
Example :

```
int a = 10;
printf("Hello");
```

If a semicolon is missing, compiler shows an error.

6. Important Points :

- The structure of a program makes it easy to read, understand and maintain.
- main() is mandatory; without it, program execution is not possible.
- #include brings library facilities; #define creates symbolic constants.
- Global variables can be used by all functions in the program.
- Every statement in C must end with a semicolon (;).
- Proper structure and indentation make programs simple and error-free.

C Programming Language.

7. Example of C Program

1. Example : "Hello World" Program

A simple C program to display a message on the screen.

```
#include <stdio.h>                  // Preprocessor directive
int main()                          // main function
{                                   // Start of main
    printf("Hello, World!\n");      // Print message
    return 0;                       // Return success
}                                   // End of main
```

2. Line by Line Explanation :

(i) `#include <stdio.h>` : Preprocessor directive. It includes standard input-output header file which contains declaration of printf().

(ii) `int main()` : Program execution starts from main() function.

(iii) `{ }` : Braces define the beginning and end of a block of code.

(iv) `printf("Hello, World!\n");` : Function to print text on the screen. "\n" is used for new line.

(v) `return 0;` : Indicates successful completion of the program. 0 means success.

(vi) `;` (Semicolon) : Every C statement must end with a semicolon.

3. Meaning of Comments in C :

Comments are ignored by the compiler. They are used to explain the code and make it easy to understand.

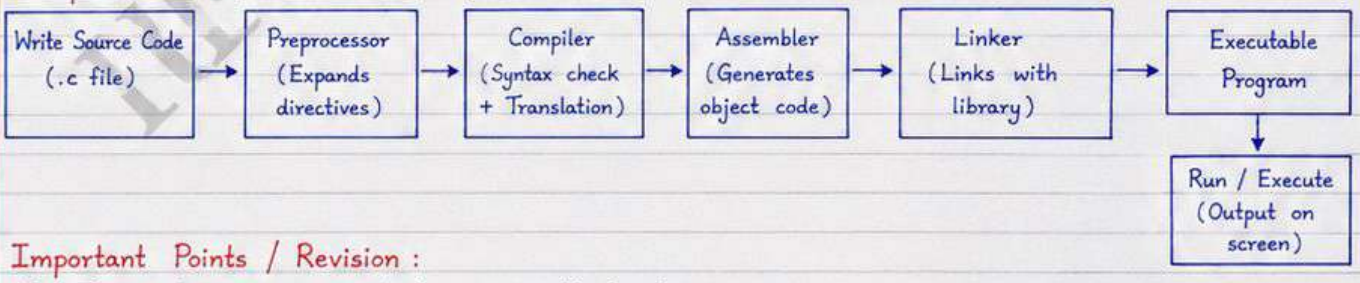
(i) Single-line Comment :
```
// This is a single-line comment
```

(ii) Multi-line Comment :
```
/*
   This is a
   multi-line comment.
   Used for longer explanation.
*/
```

4. Steps of Compilation and Execution :

① Source Code (Program written in C and saved as .c file) → ② Compile (Compiler checks for errors) → ③ Object Code (Machine code in .obj or .o file) → ④ Link (Linker combines with library files) → ⑤ Execute (Program runs and produces output)

5. Compile - Run Process (Flow) :



6. Important Points / Revision :

(i) Every C program must have main() function.

(ii) `#include <stdio.h>` is needed for input-output functions like printf(), scanf().

(iii) printf() is used to display output on the screen.

(iv) `return 0;` means the program ended successfully.

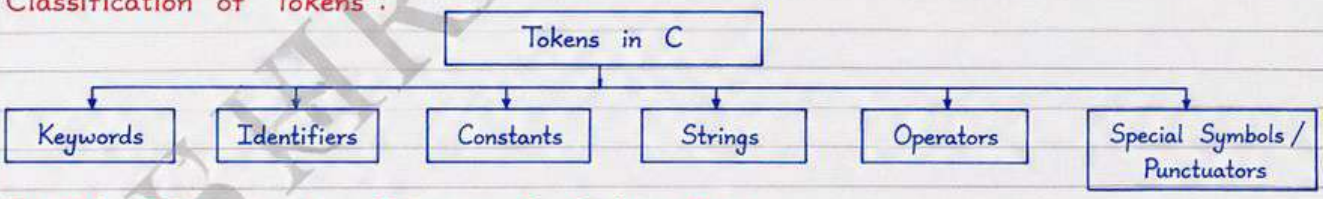
(v) Semicolon (;) is mandatory at the end of every statement.

(vi) Proper use of comments makes the program easy to read and maintain.

C Programming Language

8. Tokens in C

1. **Definition :**
   In C, a token is the smallest meaningful unit in a program that has individual meaning to the compiler.
2. **Smallest Units :**
   Tokens are the smallest individual units of a program.
3. **Types of Tokens in C :**
   C language has the following types of tokens :
   - Keywords
   - Identifiers
   - Constants
   - Strings
   - Operators
   - Special Symbols / Punctuators
4. **Explanation of Token Types :**
   (i) Keywords : Reserved words in C that have special meaning and predefined purpose. They cannot be used as identifiers.
   Examples : int, float, if, else, while, return, for, break, continue, void.
   (ii) Identifiers : Names given by the user to variables, functions, arrays, structures, etc. They help in identifying user-defined entities.
   Examples : sum, totalMarks, main, computeArea, i, j, temp1.
   (iii) Constants : Fixed values that do not change during the execution of a program.
   Examples : 10, -25, 3.14, 'A', 0, 1000L.
   (iv) Strings : Sequence of characters enclosed within double quotes (").
   Examples : "Hello", "C Language", "End of Line\n".
   (v) Operators : Symbols that perform operations on operands.
   Examples : +, -, *, /, %, =, ==, !=, <, >, <=, >=, &&, ||, !, ++, --.
   (vi) Special Symbols / Punctuators : Symbols used for grouping, separating or structuring the program.
   Examples : ( ) { } [ ] ; ; ; . . → #
5. **Classification of Tokens :**



6. **Identify Tokens in the Following C Statements :**
   (i) int sum = a + b; → { int, sum, =, a, +, b, ; }
   (ii) float avg = total / 5.0; → { float, avg, =, total, /, 5.0, ; }
   (iii) if (x > 10) { printf("Big\n"); } → { if, (, x, >, 10,), {, printf, (, "Big\n",), ;, } }
   (iv) char ch = 'A'; → { char, ch, =, 'A', ; }
   (v) count = count + 1; → { count, =, count, +, 1, ; }
7. **Important Points / Revision :**
   - Tokens are the basic building blocks of any C program.
   - The compiler breaks the program into tokens for analysis and processing.
   - Keywords have predefined meaning and cannot be redefined.
   - Identifiers follow naming rules (start with letter or '_', case-sensitive).
   - Constants may be integer, floating-point, character or enumeration.
   - Operators perform operations and punctuators provide structure to the program.
   - Correct use of tokens ensures syntactically correct and meaningful programs.

C Programming Language

9. Keywords in C

1. Meaning / Definition :

Keywords in C are special words that have a predefined meaning in the language. They are part of the syntax of C and are used to perform specific operations.

2. Important Note :

- Keywords are reserved words.
- They cannot be used as identifiers (variable names, function names, array names, etc.).

3. List of 32 C Keywords :

1. auto	9. int	17. return	25. struct
2. break	10. long	18. short	26. switch
3. case	11. register	19. signed	27. typedef
4. char	12. register	20. sizeof	28. union
5. const	13. extern	21. static	29. unsigned
6. continue	14. float	22. struct	30. void
7. default	15. for	23. switch	31. volatile
8. do	16. goto	24. typedef	32. while

4. Examples of Use of Some Important Keywords :

- int : Used to declare variables of integer type.
- if : Used to make a decision based on a condition.
- for : Used to repeat a block of code a specified number of times.
- return : Used to return a value from a function.
- while : Used to repeat a block of code as long as a condition is true.

5. Why Keywords are Important :

- They form the basic building blocks of C programs.
- Each keyword has a specific purpose and helps in controlling the flow of a program.
- Using keywords correctly makes the program meaningful and error-free.

6. Revision Points :

(i) Keywords are reserved and have predefined meaning.
(ii) They cannot be used as identifiers.
(iii) There are 32 keywords in C.
(iv) Keywords are essential for writing correct and efficient C programs.

C Programming Language.

10. Identifiers in C

1. **Definition :**

   An identifier is a user-defined name given to program elements such as variables, functions, arrays, structures, labels, etc., to identify them uniquely in a C program.

2. **Rules for Naming Identifiers :**

   - An identifier may contain letters (A-Z, a-z), digits (0-9) and underscore (_).
   - The first character must be a letter (A-Z or a-z) or an underscore (_).
   - It cannot start with a digit.
   - No spaces or special symbols are allowed except underscore (_).
   - Identifiers are case sensitive. (sum, Sum and SUM are different.)
   - It should not be a C keyword or reserved word.
   - Length of an identifier can be implementation dependent, but the first 31 characters are significant in most compilers.

3. **Types of Identifiers (with Examples) :**

   - Variable names : age, total, _count, price1
   - Function names : add(), computeSum(), printData()
   - Array names : marks, scores, arr1
   - Structure names : Student, Employee, Complex
   - Other identifiers : label1, MAX_SIZE, temp_value

4. **Valid and Invalid Identifier Examples :**

Valid Identifiers	Invalid Identifiers
• count	• 1count (starts with digit)
• _total	• total sum (contains space)
• student1	• @value (special character not allowed)
• sum_value	• float (keyword)
• MAX_SIZE	• a+b (special character)
• getData	• 2_value (starts with digit)
• Value123	• int (keyword)

5. **Naming Conventions and Good Practices :**

   - Use meaningful names that describe the purpose of the identifier.
   - Use lowercase letters for variable and function names (e.g., total, calculateSum).
   - Use uppercase letters for constants (e.g., MAX, PI).
   - Use underscores to improve readability (e.g., total_marks, student_age).
   - Avoid using single-letter names except in loop counters (i, j, k).
   - Keep names short but descriptive.

6. **Keywords vs. Identifiers :**

   - Keywords are reserved words in C that have a predefined meaning (e.g., int, if, return, for, while).
   - Identifiers are user-defined names given to program elements.
   - Example : 'int' is a keyword and cannot be used as an identifier. But 'integer' is a valid identifier.

7. **Short Note – Identifiers and Variables :**

   Variables are named memory locations used to store data values. In C, each variable must have a unique identifier so that the compiler can recognize and access the correct memory location.

8. **Important Points / Revision :**

   (i) Identifiers are names given to program elements like variables, functions, arrays, etc.
   (ii) They must follow specific rules and should not be keywords.
   (iii) Good identifier names make programs easy to read, understand and maintain.

C Programming Language.

21. Operators in C – Introduction and Types

1. Definition :

An operator is a special symbol that tells the compiler to perform specific mathematical, logical, or bit-level operations on one or more operands and produce a result.

2. Importance of Operators :

- Operators are the building blocks of expressions in C.
- They are used to perform calculations, comparisons, and logical decisions.
- Operators help in writing compact, efficient, and readable programs.

3. Operands :

- The values or variables on which an operator works are called operands.
- An operator may work on one (unary), two (binary), or three operands. (ternary, e.g., conditional operator).

4. Types of Operators in C :

C provides a rich set of operators which can be classified as follows:

(i) Arithmetic Operators
(ii) Relational (Comparison) Operators
(iii) Logical Operators
(iv) Assignment Operators
(v) Increment and Decrement Operators
(vi) Conditional (Ternary) Operator
(vii) Bitwise Operators
(viii) Special Operators

5. Explanation of Operators :

(i) Arithmetic Operators : Used to perform mathematical operations such as addition, subtraction, multiplication, division, modulus.
Operators : +, −, *, /, %

(ii) Relational Operators : Used to compare two operands. The result is either true (1) or false (0).
Operators : <, <=, >, >=, ==, !=

(iii) Logical Operators : Used to combine two or more conditions.
Operators : && (AND), || (OR), ! (NOT)

(iv) Assignment Operators : Used to assign a value to a variable.
Operators : =, +=, −=, *=, /=, %=, <<=, >>=, &=, ^=, |=

(v) Increment/Decrement Operators : Used to increase or decrease the value of a variable by 1.
Operators : ++ (increment), −− (decrement)

(vi) Conditional (Ternary) Operator : A shorthand if-else operator.
Syntax : condition ? expression1 : expression2;

(vii) Bitwise Operators : Used to perform operations at the bit level.
Operators : &, |, ^, ~, <<, >>

(viii) Special Operators : Miscellaneous operators used in C.
Operators : sizeof, comma (.), address (&), dereference (*), subscript ([]), member access (.), function call ()

6. Important Revision Points :

- Operators work on operands and produce a result.
- Operator precedence and associativity decide the order of evaluation in expressions.
- Use parentheses ( ) to override precedence and make expressions clear.
- Relational and logical operators return 1 for true and 0 for false.
- ++ and −− can be prefix or postfix; their position affects the result.
- Bitwise operators are useful for low-level programming and optimization.
- Ternary operator helps in writing compact conditional statements.
- Always use operators carefully to avoid logic errors and unexpected results.

C Programming Language.

22. Arithmetic Operators and Expressions

1. Meaning of Arithmetic Operators :

Arithmetic operators are used to perform mathematical operations on numeric data (integers or real numbers).

2. Arithmetic Operators in C :

Operator	Name	Meaning	Example	Result
+	Addition	Adds two operands	a + b	Sum of a and b
-	Subtraction	Subtracts right operand from left	a - b	Difference of a and b
*	Multiplication	Multiplies two operands	a * b	Product of a and b
/	Division	Divides left operand by right	a / b	Quotient of a / b
%	Modulus	Remainder after integer division	a % b	Remainder of a / b

3. Unary Plus and Unary Minus :

- Unary plus (+) keeps the value unchanged.
- Unary minus (-) changes the sign of the operand.

Examples :

+a is same as a

-a is negative of a

Examples :
- If a = 7, +a = 7
- If a = 7, -a = -7
- -(-a) = a
- +(-a) = -a

4. Integer Division vs Real Division :

- If both operands are integers, the result of '/' is also an integer (fractional part discarded).
- If any one operand is real (float or double), the result is real.
- To get real division, at least one operand must be real.

Examples :

Integer Division		Real Division
7 / 2 = 3	// 7 ÷ 2 = 3.5, but result is 3	7.0 / 2 = 3.5
7 % 2 = 1	// remainder is 1	7 / 2.0 = 3.5
		7.0 / 2.0 = 3.5

5. Precedence of Arithmetic Operators : (Higher precedence evaluated first)

Precedence (High to Low)	Operators	Associativity
1	*, /, %	Left to Right
2	+, -	Left to Right

Note : Operators with same precedence are evaluated from left to right.

6. Arithmetic Expressions :

- An expression is a combination of operands (constants, variables) and operators.
- It may contain any number of operators and operands.
- Parentheses '(' ')' can be used to change the natural precedence.

7. Solved Examples (with Output) :

Example / Expression	Explanation	Output
(i) a = 10 + 5 * 2;	5 * 2 = 10, then 10 + 10 = 20	a = 20
(ii) b = (10 + 5) * 2;	(10 + 5) = 15, then 15 * 2 = 30	b = 30
(iii) c = 20 / 3;	Integer division: 20 ÷ 3 = 6 remainder 2	c = 6
(iv) d = 20 % 3;	Remainder when 20 is divided by 3	d = 2
(v) e = -5 + +3 * -2;	+3 * -2 = -6, then -5 + (-6) = -11	e = -11

8. Important Points / Revision :

- Arithmetic operators work on numeric data types (int, float, double).
- Division by zero is not allowed (run-time error).
- The '%' operator works only with integer operands.
- Use parentheses to make expressions clear and to change the order of evaluation.
- Always remember operator precedence to avoid mistakes.
- Arithmetic expressions are widely used in programs, formulas and computations.

C Programming Language

23. Relational and Logical Operators

1. Relational Operators :

Relational operators are used to compare two operands. The result is either true (1) or false (0). They are used in conditions of decision making and loops.

Operator	Meaning	Example	True When
<	Less than	a < b	a is smaller than b
>	Greater than	a > b	a is greater than b
<=	Less than or equal to	a <= b	a is less than or equal to b
>=	Greater than or equal to	a >= b	a is greater than or equal to b
==	Equal to	a == b	a is equal to b
!=	Not equal to	a != b	a is not equal to b

2. Logical Operators :

Logical operators are used to combine two or more conditions. The result is either true (1) or false (0).

Operator	Meaning	Use	Example
&&	Logical AND	True if both conditions are true	(a > 5 && b < 10)
||	Logical OR	True if any one condition is true	(a < 5 || b == 0)
!	Logical NOT	True if the condition is false	!(a == b)

3. Truth Values :

- True is represented by 1.
- False is represented by 0.
- In conditions, any non-zero value is treated as true.
- Zero is treated as false.

4. Truth Table for Logical Operators :

(a) Logical AND (&&)

A	B	A && B
0	0	0
0	1	0
1	0	0
1	1	1

(b) Logical OR (||)

A	B	A || B
0	0	0
0	1	1
1	0	1
1	1	1

(c) Logical NOT (!)

A	!A
0	1
1	0

5. Examples in Conditions :

- if (a > b) // True if a is greater than b
- if (a <= b && b != 0) // True if a is less or equal to b and b is not zero
- if (marks >= 40 || bonus > 0) // True if marks >= 40 or bonus is greater than 0
- if (!(x == y)) // True if x is not equal to y

6. Difference Between Relational and Logical Operators :

Basis	Relational Operators	Logical Operators
Purpose	Compare two operands.	Combine two or more conditions.
Operands	Work on values (int, float, char, etc.)	Work on conditions (true/false).
Result	Returns true (1) or false (0).	Returns true (1) or false (0).
Examples	<, >, <=, >=, ==, !=	&&, ||, !
Used In	Conditions, expressions.	Complex conditions, decision making.

7. Important Revision Points :

- Relational operators compare values and return 1 (true) or 0 (false).
- Logical operators combine conditions and return 1 (true) or 0 (false).
- In C, 0 is false and any non-zero value is true.
- Use parentheses to make complex conditions clear.
- Relational operators have higher precedence than logical operators.
- Avoid using assignment (=) in place of equality operator (==) in conditions.
- Practice writing conditions using combinations of &&, || and !.

C Programming Language.

24. Assignment, Increment and Decrement Operators

1. Assignment Operator (=) :

- Used to assign a value to a variable.
- The right-hand value is evaluated first, then assigned to the left-hand variable.
- It is a binary operator and right-associative.

Example :

```
int a;
a = 10;        // a now contains 10
```

2. Compound Assignment Operators :

Operator	Meaning	Example	Equivalent To
+=	Add and assign	a += 5;	a = a + 5;
-=	Subtract and assign	a -= 3;	a = a - 3;
*=	Multiply and assign	a *= 2;	a = a * 2;
/=	Divide and assign	a /= 4;	a = a / 4;
%=	Modulus and assign	a %= 3;	a = a % 3;

3. Increment (++) and Decrement (--) Operators :

- ++ increases the value of operand by 1.
- -- decreases the value of operand by 1.
- Can be used in two forms: prefix and postfix.

4. Prefix vs Postfix :

Form	Meaning	Example (int x = 5;)	Operation	Value Used in Expression	Final Value of x
Prefix ++x	Increment then use	y = ++x;	x becomes 6 first	6	6
Postfix x++	Use then increment	y = x++;	x used as 5, then x becomes 6	5	6
Prefix --x	Decrement then use	y = --x;	x becomes 4 first	4	4
Postfix x--	Use then decrement	y = x--;	x used as 5, then x becomes 4	5	4

5. Examples (Short Programs) :

(ii)
```
int a = 5, b;
b = ++a;
printf("a = %d, b = %d\n", a, b);
// Output:
// a = 6, b = 6
```

(iii)
```
int a = 5, b;
b = a++;
printf("a = %d, b = %d\n", a, b);
// Output:
// a = 6, b = 5
```

(iii)
```
int a = 5, b;
b = --a + a--;
printf("a = %d, b = %d\n", a, b);
// Step 1: --a => a = 4
// Step 2: use a (4) in +
// Step 3: a-- => a = 3
// Output: a = 3, b = 8
```

6. Common Mistakes :

- Confusing prefix and postfix forms.
- Writing expressions like: i = ++i++; // Invalid
- Using = instead of == in conditions (logical error).
- Modifying a variable multiple times in one expression without understanding sequence.

7. Important Revision Points :

- Use = to assign a value; it does not compare.
- Compound epresstors save typing and make code clean.
- Prefix changes value first; postfix uses value first.
- In complex expressions, order of evaluation is left-to-right.
- Avoid changing the same variable more than once in a single expression.
- Always test expressions with small examples to predict output.
- Good use of ++ and -- improves efficiency in loops and programs.

C Programming Language.

25. Conditional, Bitwise and Special Operators

1. Conditional (Ternary) Operator ? :

- Used to make decisions and return one of two values.
- It is a shorthand form of if-else.

Syntax : `condition ? expression_if_true : expression_if_false;`

Example :
```
int max = (a > b) ? a = b;          // returns a if a > b, else b
char grade = (marks >= 50) ? 'P' : 'F';   // pass or fail
```

2. Bitwise Operators :

Operator	Name	Description (Meaning)	Example (a = 5, b = 3)	Result
&	Bitwise AND	1 if both bits are 1	a & b	1 (0001)
|	Bitwise OR	1 if any one of the bits is 1	a | b	7 (0111)
^	Bitwise XOR	1 if bits are different	a ^ b	6 (0110)
~	Bitwise NOT	Inverts all bits (1's complement)	~a	~5 = -6*
<<	Left Shift	Shifts bits left by n positions (fills 0)	a << 1	10 (1010)
>>	Right Shift	Shifts bits right by n positions	a >> 1	2 (0010)

* For signed integers, ~x = -(x+1) when using two's complement.

3. Special Operators :

- **sizeof operator** : Returns the size (in bytes) of data type / variable.
  Syntax : sizeof (data_type or variable)
  Examples : sizeof (int); sizeof (float); sizeof x;
- **Comma operator , :** Allows two or more expressions to be evaluated in sequence.
  Syntax : expression1, expression2, ..., expressionN
  The value of the whole expression is the value of the last expression.
  Example : `int a = 5, b = 10, c;  c = (a++, b++, a + b);  // c = 16, a = 6, b = 11`
- **Address-of operator &** : Returns the memory address of a variable.
  Example : `int x = 10;  int *p = &x;  // &x gives address of x`
- **Indirection (Value-at) operator *** : Returns the value stored at the given address.
  Example :
  ```
  int x = 10;  int *p = &x;
  printf("%d", *p);  // prints 10
  ```
- **Other useful operators :**
  ( ) Parentheses for grouping and function call.
  [ ] Array subscript.
  . Member access (structure/union).
  -> Member access through pointer to structure.

4. Summary Table :

Operator Type	Operators	Purpose	Example
Conditional	? :	Selects value based on condition	x = (a > b) ? a : b;
Bitwise	&, |, ^, ~, <<, >>	Performs operations on bits	a & b, a << 1, ~a
Special	sizeof, , , &, *	Size, sequence, address and value at address	sizeof(x), x, &x, *p

5. Important Revision Points :

- Conditional operator is the only operator that takes three operands.
- Bitwise operators work on individual bits, not on whole numbers.
- Left shift (<<) multiplies by 2^n; Right shift (>>) divides by 2^n (for unsigned).
- ~ is unary operator (1's complement).
- sizeof does not evaluate its operand (no side effects).
- Comma operator evaluates left to right; value of last expression is returned.
- & gives address of variable; * gives value stored at that address.
- Use parentheses to avoid ambiguity in complex expressions.

C Programming Language.

26. Decision Making in C – Introduction

1. Meaning of Decision Making / Selection Statements :

- Decision making means allowing the program to choose different actions based on different conditions.
- It is also called selection or branching.
- The program flow changes according to whether a condition is true or false.

2. Need for Conditions :

- Every problem has situations where action depends on some condition.
- Conditions make programs logical and flexible.
- They help in comparing values and taking appropriate decisions.

3. Flow of Control :

- Normally, a C program executes statements sequentially.
- With decision making, the flow can bifurcate (branch) and then merge.
- The control goes to the block of statements whose condition is satisfied.

4. Boolean / True – False Concept :

- Conditions in C evaluate to either true (1) or false (0).
- Any non-zero value is considered true; zero is considered false.
- This is the basis of all decision making in C.

5. Decision-Making Statements in C :

①	if statement	: Executes a block if condition is true.
②	if – else statement	: Executes one block if true, another if false.
③	Nested if statement	: An if or if-else inside another if or else.
④	else-if ladder	: Used to test multiple conditions in sequence.
⑤	switch statement	: Used to select one block from many cases based on the value of an expression.
⑥	Conditional operator (? :)	: A shorthand if-else used for simple decisions.

6. General Syntax Forms :

- if — `if (condition) { statements; }`
- if – else — `if (condition) { statements; } else { statements; }`
- else-if ladder — `if (cond1) { ... } else if (cond2) { ... } ... else { ... }`
- switch — `switch (expression) { case value1: ... ; break; case value2: ... ; break; ... default: ... }`
- conditional operator — `(condition) ? expression1 : expression2;`

7. Example (Simple if-else) :

```
int x;
scanf("%d", &x);
if (x >= 0)
    printf("Positive");
else
    printf("Negative");
```

→ If x is 0 or more, prints "Positive".

→ Otherwise, prints "Negative".

8. Important Revision Points :

- Decision making is based on conditions that evaluate to true (1) or false (0).
- Use relational operators (>, <, >=, <=, ==, !=) to form conditions.
- Use logical operators (&&, ||, !) to combine conditions.
- Incomplete / wrong conditions cause logical errors.
- Always use braces {} for multi-statement blocks to avoid ambiguity.
- Nested and else-if statements improve program logic.
- Switch is faster and clearer for multiple choices on integral or character values.
- Conditional operator (? :) is useful for short, simple decisions.
- Test all possible cases and boundary values while writing conditions.
- Good indentation and meaningful variable names improve readability and avoid mistakes.

C Programming Language

27. if Statement in C

1. Introduction :

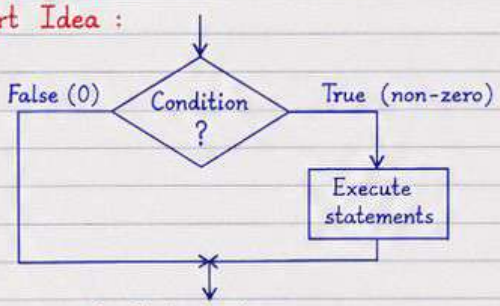
The if statement is used to execute a block of statements only when a specified condition is true.

2. Syntax :

```
if ( condition )
{
    // statements;
}
```

→ condition is any valid expression.
→ If condition is true (non-zero), statements inside the block are executed.

3. Flowchart Idea :



- If the condition is true, the statements inside if block are executed.
- If the condition is false, the statements inside if block are skipped.
- After execution, control moves to the next statement after the if block.

4. Condition in if Statement :

- The condition can be any relational or logical expression.
- In C, 0 is considered False and any non-zero value is True.

5. Examples of Simple if Programs :

(i) Check if a number is positive

```
#include <stdio.h>
int main()
{   int n;
    printf("Enter a number: ");
    scanf("%d", &n);
    if (n > 0)
        printf("\nThe number is positive.\n");
    return 0;
}
```

Example Output :
Enter a number: 7
The number is positive.

(ii) Check if a number is even

```
#include <stdio.h>
int main()
{   int n;
    printf("Enter a number: ");
    scanf("%d", &n);
    if (n % 2 == 0)
        printf("\nThe number is even.\n");
    return 0;
}
```

Example Output :
Enter a number: 12
The number is even.

(iii) Check eligibility for voting (age >= 18)

```
#include <stdio.h>
int main()
{   int age;
    printf("Enter your age: ");
    scanf("%d", &age);
    if (age >= 18)
        printf("\nYou are eligible to vote.\n");
    return 0;
}
```

Example Output :
Enter your age: 20
You are eligible to vote.

6. Important Points :

- The condition in if must be inside parentheses ( ).
- If condition is true, only the statements inside the if block are executed.
- If condition is false, the if block is skipped.
- An if statement can have zero or more statements inside the block.
- If multiple statements are written without braces { }, only the first statement controlled by if (others execute always).
- if statement is the building block of decision making in C.

C Programming Language

28. if-else Statement and Nested if

1. if-else Statement :

- The if-else statement is used to execute one block of code if a condition is true and another block if the condition is false.

2. Syntax of if-else Statement :

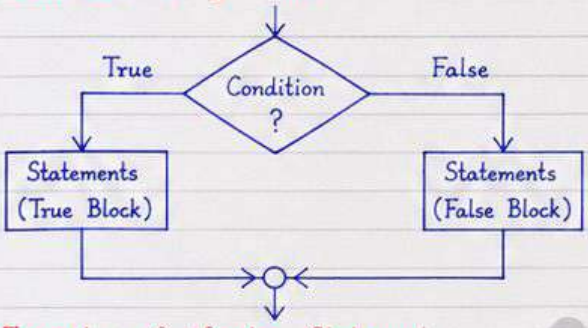
```
if (condition)
{
    // block of statements if condition is true
}
else
{
    // block of statements if condition is false
}
```

Here, condition is any relational or logical expression that results in true (non-zero) or false (0).

3. When to Use if-else ?

- When there are two possible choices.
- To make decisions based on a condition.
- To control the flow of program execution.

4. Flowchart of if-else :



5. Examples of if-else Statement :

(i) Check whether a number is positive or negative.

```
int n;
scanf("%d", &n);
if (n >= 0)
    printf("%d is positive.\n", n);
else
    printf("%d is negative.\n", n);
```

(ii) Check whether a number is even or odd.

```
int n;
scanf("%d", &n);
if (n % 2 == 0)
    printf("%d is even.\n", n);
else
    printf("%d is odd.\n", n);
```

6. Nested if : Concept

- Sometimes, we need to check another condition inside a block of if or else.
- Such a situation is called nested if.
- if-else inside another if or else.
- It allows decision making at multiple levels.

7. Syntax of Nested if :

```
if (condition1)
{
    if (condition2)
    {
        // statements
    }
    else
    {
        // statements
    }
}
else
{
    // statements
}
```

8. Examples of Nested if :

(i) Find the greatest of two numbers.

```
int a, b;
scanf("%d %d", &a, &b);
if (a > b)
    printf("%d is greater.\n", a);
else if (b > a)
    printf("%d is greater.\n", b);
else
    printf("Both numbers are equal.\n");
```

(ii) Find the greatest of three numbers.

```
int a, b, c, max;
scanf("%d %d %d", &a, &b, &c);
if (a >= b)
{
    if (a >= c) max = a;
    else        max = c;
}
else
{
    if (b >= c) max = b;
    else        max = c;
}
printf("Greatest = %d\n", max);
```

9. Important Points / Revision :

- if-else is a two-way decision statement.
- The condition in if must be in parentheses and should be true (non-zero) or false (0).
- if and else blocks can contain single or multiple statements.
- Use braces { } to avoid ambiguity, even for single statements.
- Nested if increases program logic capability; keep indentation and brackets clear.
- Never use assignment (=) in place of comparison (==) in conditions.
- else corresponds to the nearest unmatched if.

C Programming Language

29. else-if Ladder in C

1. Purpose of else-if Ladder :

- To test multiple conditions where only one block of code needs to execute.
- It is an efficient way to handle multiple choices.
- It improves readability compared to nested if statements.

2. Syntax of else-if Ladder :

```
if (condition1)
    statement(s);
else if (condition2)
    statement(s);
else if (condition3)
    statement(s);
  .
  .
  .
else
    statement(s);
```

- The conditions are checked from top to bottom.
- As soon as one condition is true, its block executes and the rest are skipped.
- else part is optional.

3. Control Flow of else-if Ladder :

- Check the first condition (condition1).
- If true → execute its block and skip the remaining.
- If false → check the next condition (condition2).
- Continue until a condition is true.
- If none are true and else exists → execute else block.
- If no condition is true and no else part → no block executes.

4. Examples :

(i) Grading System

```
#include <stdio.h>
int main()
{
   int marks;
   printf("Enter marks (0-100): ");
   scanf("%d", &marks);
   if (marks >= 90)
       printf("Grade : O\n");
   else if (marks >= 75)
       printf("Grade : A\n");
   else if (marks >= 60)
       printf("Grade : B\n");
   else if (marks >= 40)
       printf("Grade : C\n");
   else
       printf("Grade : F\n");
   return 0;
}
```

Example Output :

```
Enter marks (0-100): 82
Grade : A
-----------------
Enter marks (0-100): 55
Grade : C
-----------------
Enter marks (0-100): 33
Grade : F
```

(ii) Menu Driven Selection

```
#include <stdio.h>
int main()
{
   int ch;
   printf("\nMenu:\n");
   printf("1. Add\n2. Subtract\n3. Multiply\n4. Divide\n");
   printf("Enter your choice (1-4): ");
   scanf("%d", &ch);
   if (ch == 1)
      printf("Addition selected\n");
   else if (ch == 2)
      printf("Subtraction selected\n");
   else if (ch == 3)
      printf("Multiplication selected\n");
   else if (ch == 4)
      printf("Division selected\n");
   else
      printf("Invalid choice\n");
   return 0;
}
```

Example Output :

```
Menu:
1. Add
2. Subtract
3. Multiply
4. Divide
Enter your choice (1-4): 3
Multiplication selected
-----------------
Menu:
1. Add
2. Subtract
3. Multiply
4. Divide
Enter your choice (1-4): 5
Invalid choice
```

5. Difference Between if-else and else-if Ladder :

Aspect	if-else Statement	else-if Ladder
Number of conditions	Works with only two conditions	Works with multiple conditions
Structure	Single selection (two-way)	Multiple selection (multi-way)
Readability	Less readable for many conditions	More readable and organized
Efficiency	May require nested if-else	Requires single sequence
Use	Simple decisions	Complex decisions and menus

6. Important Revision Points :

- else-if ladder executes only one block of code.
- Conditions are checked in order from top to bottom.
- As soon as a condition is true, the remaining conditions are skipped.
- Ensure conditions are mutually exclusive to avoid logic errors.
- Use else to handle default cases.
- Always test all possible inputs (boundary values).
- Proper indentation improves readability.
- Avoid too many conditions; break logic into functions if required.
- Comment your code for clarity.